Relicarium II

Actividad Integradora 2

Reingeniería, gestión de servicios nativos

y auditoría de código en repositorio público

Jose Andres Mendoza Castro

Desarrollo de Aplicaciones Móviles

Docente: Gregorio Manuel Cruz Gonzabay

1 de noviembre de 2026

Continuidad del proyecto. Este trabajo utiliza la aplicación RELICARIUM de la Actividad Integradora 1 y documenta las mejoras realizadas para su evolución a Relicarium II.

Sección I

Justificación del rebranding y cuadro comparativo

Relicarium II conserva la temática de artefactos legendarios de la primera entrega y evoluciona de un catálogo de consulta a una colección personal con fotografías del usuario. La nueva propuesta de valor combina exploración visual, disponibilidad offline y acceso a servicios del dispositivo. Los productos y precios son ficticios; no se implementan compras ni pagos.

Se mantuvo la raíz Relicarium por decisión del estudiante y se añadió II para identificar la nueva etapa. La identidad usa un relicario azul luminoso sobre negro profundo y una interfaz oscura con acentos celestes. La Tabla 1 resume el cambio y la Figura 1 permite contrastar las pantallas.

Tabla 1

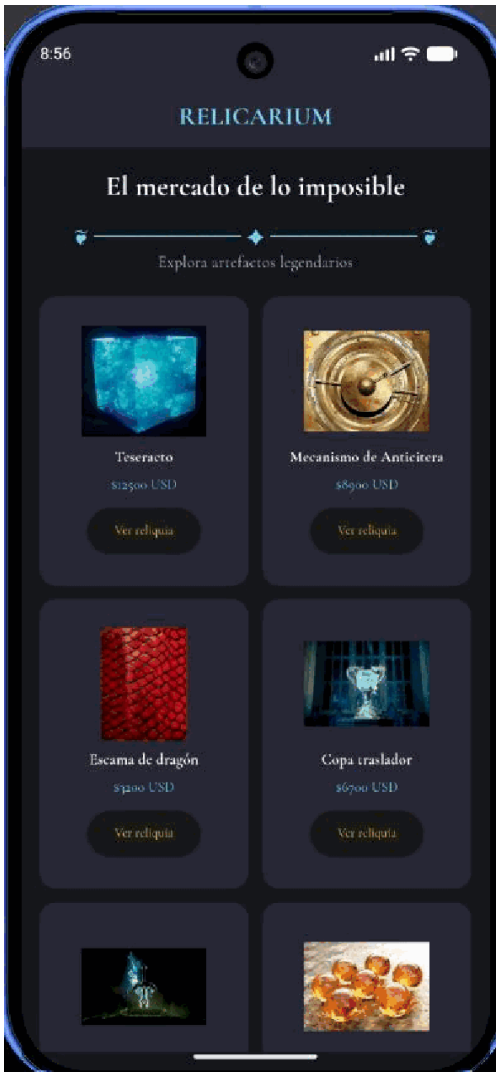
Comparación de la primera entrega y la reingeniería

Elemento	Actividad 1	Actividad 2
Marca	RELICARIUM	Relicarium II
Identificador Android	com.example.tech_catalog	com.josemendoza.relicarium2
Interfaz	Tarjetas con nombre, precio y botón Ver reliquia	Tarjeta completa pulsable; precio en el detalle
Tema	Oscuro con celeste y controles ámbar	Oscuro y celeste; borde fino y textos destacados
Icono y splash	Recursos iniciales del proyecto	Relicario personalizado y fondo nativo oscuro en Android
Servicios	Consulta de datos incluidos en Home	Red reactiva, catálogo persistido, fotos y alertas

Figura 1

Evolución visual del catálogo

Antes: RELICARIUM



Después: Relicarium II



Nota. La imagen izquierda procede de la entrega de la Actividad Integradora 1. La derecha es una captura de la segunda etapa con el banner offline activo. Las imágenes mantienen su proporción; las diferencias de estado se señalan para no confundirlas con cambios puramente visuales.

En la segunda interfaz, la imagen ocupa la mayor parte de la tarjeta. Al eliminar el botón individual y el precio del catálogo, se refuerza el carácter de galería. El texto invita a tocar la reliquia; el detalle conserva nombre, descripción y precio del mismo objeto Product.

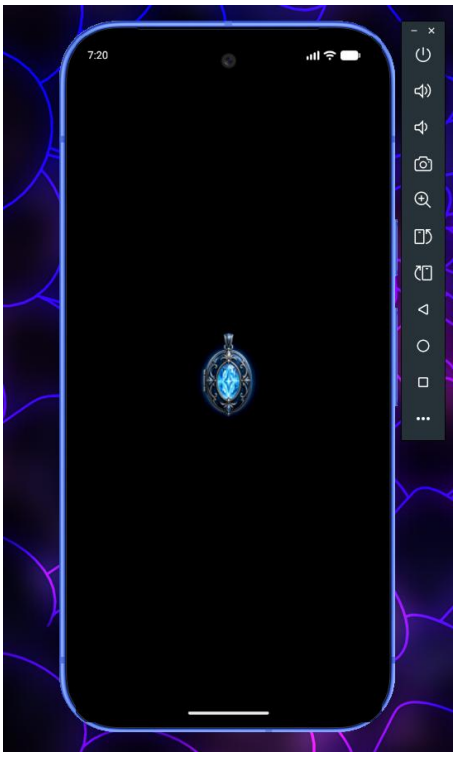
Figura 2

Recurso de identidad y splash de Android

Recurso gráfico del icono



Splash en ejecución



Nota. El recurso del icono fue generado con asistencia de IA. La imagen derecha muestra el splash nativo real del emulador. Se optó por una pantalla estática; no se implementó animación. El catálogo se muestra al terminar el arranque.

La paleta relaciona el azul luminoso del símbolo con los títulos y bordes celestes de la interfaz. Los fondos oscuros dan protagonismo a los artefactos. El icono y el splash Android se generaron a partir de configuraciones versionadas, en lugar de reemplazar manualmente cada resolución.

La Figura 2 distingue un recurso de diseño de una evidencia de ejecución. Esta separación permite conservar la trazabilidad de los materiales visuales sin presentar una ilustración como si fuera una prueba del dispositivo.

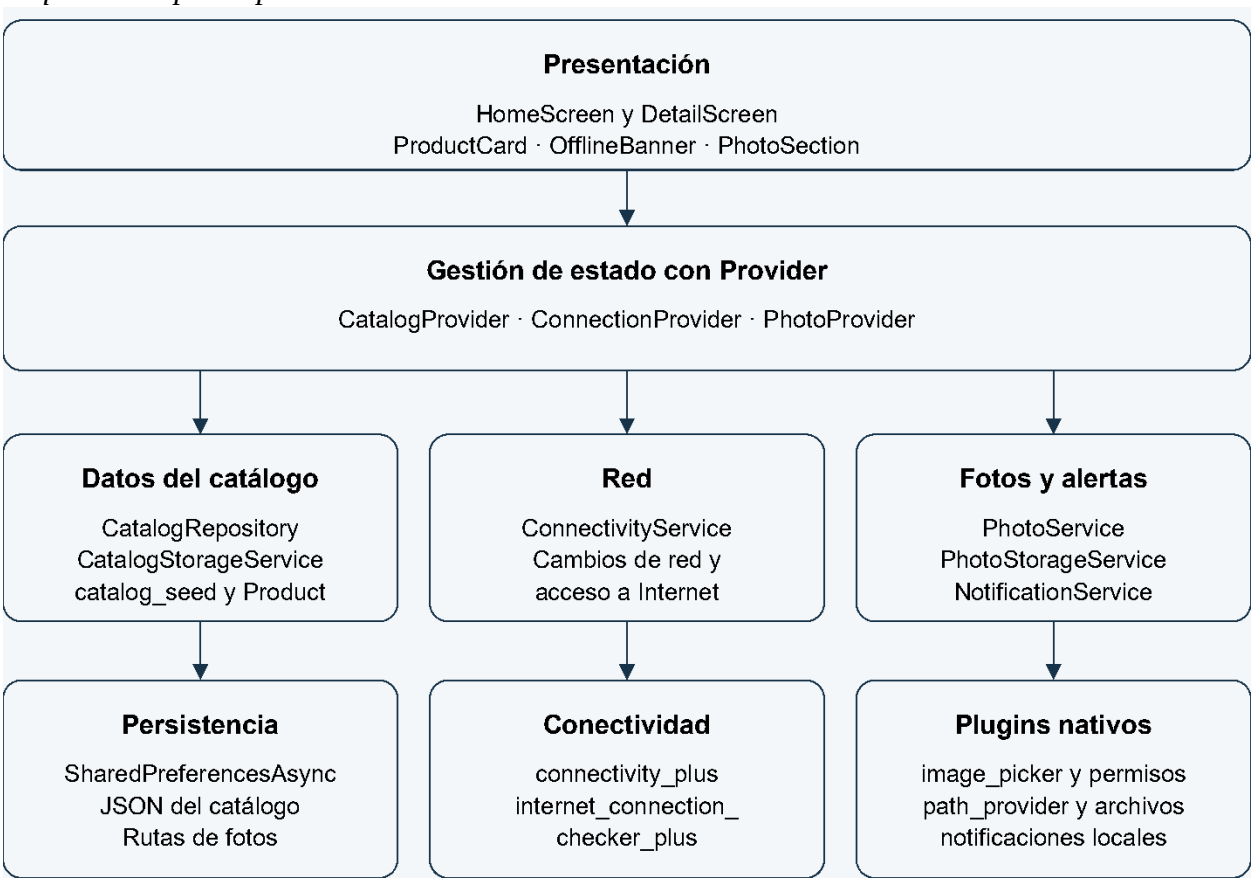
Sección II

Arquitectura y patrones de desarrollo

La arquitectura separa interfaz, estado, datos y servicios. Los widgets observan Providers y delegan las operaciones nativas. Este patrón de estado observable se basa en ChangeNotifier (Flutter, s. f.-c). La Figura 3 representa las dependencias principales.

Figura 3

Arquitectura por capas de Relicarium II



Nota. Diagrama elaborado a partir del código del repositorio. Las flechas indican dependencia o delegación de operaciones; los plugins pueden utilizar mecanismos nativos o verificaciones de red según su función.

CatalogProvider y ConnectionProvider se suministran sobre la aplicación. El PhotoProvider se crea en cada detalle con la clave de la reliquia. Así, las fotografías personales se asocian al producto correcto y la interfaz no solicita permisos ni escribe archivos directamente.

Responsabilidades y flujo de datos

Product representa nombre, descripción, precio e identificador del recurso de imagen, y ofrece conversión a JSON. CatalogRepository consulta la copia persistida mediante CatalogStorageService. Si no existe una copia válida, utiliza catalog_seed como inicialización y la guarda. Home observa CatalogProvider y muestra carga, error o cuadrícula.

La persistencia se implementa con SharedPreferencesAsync, adecuada aquí para una colección pequeña de datos no críticos. El catálogo no procede de un servidor: se guarda una copia de los datos iniciales y se recupera al reabrir la aplicación. Las ilustraciones continúan empaquetadas como assets; las fotos personales se guardan como archivos privados (Flutter, s. f.-b).

Ciclo de vida y concurrencia

ConnectionProvider escucha dos streams y comprueba Internet al regresar la aplicación al primer plano. Un contador de revisión evita que una respuesta antigua sobrescriba una comprobación reciente. En dispose se retira el observador y se cancelan las suscripciones.

PhotoProvider expone estados de carga, error y previsualización. Su bandera busy evita abrir selectores o guardar simultáneamente desde el mismo detalle. Las operaciones asíncronas comprueban si el Provider fue destruido antes de notificar cambios. El guardado de archivos y el envío de alertas manejan sus errores por separado.

Alcance de plataformas

La validación funcional se realizó en Android. Se prepararon nombre, bundle identifier, descripciones de cámara/galería y delegado de notificaciones en iOS, además de la configuración Darwin del servicio. La compilación, los ajustes restantes de plugins y la ejecución en iOS no se validaron en un Mac; no se afirma soporte completo de esa plataforma.

Sección III

Explicación de código y manejo de excepciones

Detección reactiva y acceso efectivo a Internet

Archivo: lib/services/connectivity_service.dart. Se muestran los accesos a los streams y la comprobación puntual.

```
Stream<List<ConnectivityResult>> get networkChanges =>
    _network.onConnectivityChanged;

Stream<bool> get internetChanges => _internet.onStatusChange
    .map((status) => status == InternetStatus.connected)
    .distinct();

Future<bool> checkInternet() async {
  try {
    return await _internet.hasInternetAccess.timeout(
      const Duration(seconds: 8),
    );
  } catch (_) {
    return false;
  }
}
```

networkChanges detecta cambios de las interfaces del dispositivo. **internetChanges** convierte los estados del verificador en un valor booleano y evita repetir valores iguales. Esta distinción es necesaria porque tener Wi-Fi no garantiza que exista salida a Internet (Flutter Community, s. f.; Internet connection checker plus, s. f.).

checkInternet utiliza await para esperar sin bloquear la interfaz. El timeout limita la espera a ocho segundos; si ocurre un error, devuelve false. ConnectionProvider comienza con estado null, equivalente a comprobando, y OfflineBanner solo aparece cuando el estado es false. Al reconectar, el cambio de estado reconstruye el widget y retira el aviso.

Persistencia y recuperación del catálogo

Archivo: lib/repositories/catalog_repository.dart. Fragmento del método loadProducts.

```
final saved = await _storage.loadProducts();

if (saved != null) {
  debugPrint('Catalogo recuperado del almacenamiento local.');
  return saved;
}

await _storage.saveProducts(initialProducts);
debugPrint('Catalogo inicial guardado en almacenamiento local.');
return List<Product>.of(initialProducts);
```

El repositorio intenta recuperar primero la copia local. Solo si el resultado es null escribe los datos iniciales. CatalogStorageService serializa los productos con toJson y los reconstruye con fromJson. Captura FormatException y TypeError cuando el contenido no es válido; el repositorio utiliza los datos iniciales como respaldo ante PlatformException.

Permisos solicitados durante el uso

Archivo: lib/services/photo_service.dart. Fragmento previo a la apertura de la cámara.

```
if (fromCamera) {
  final permission = await Permission.camera.request();
  if (!permission.isGranted) {
    throw PhotoPermissionException(
      permanentlyDenied: permission.isPermanentlyDenied,
    );
  }
}
```

El permiso se solicita cuando se pulsa Cámara. Si se deniega, se lanza una excepción propia que PhotoProvider traduce en un mensaje; si queda bloqueado, se ofrece Abrir ajustes. La galería utiliza el selector del sistema sin solicitar acceso general a los archivos (Baseflow, s. f.).

Captura, procesamiento, pre visualización y guardado

Archivo: lib/services/photo_service.dart. Fragmento de selección y procesamiento solicitado.

```
final photo = await _picker.pickImage(
  source: fromCamera ? ImageSource.camera : ImageSource.gallery,
  maxWidth: 1600,
  maxHeight: 1600,
  imageQuality: 75,
  requestFullMetadata: false,
);
return photo?.path;
```

source elige cámara o galería. Se solicita limitar las dimensiones a 1600 por 1600 y usar calidad 75; el efecto depende del formato y de la implementación del plugin. No se implementó un editor de recorte manual ni se presenta una medición de reducción de bytes. Al cancelar, photo es null y se conserva la selección anterior (Flutter, s. f.-a).

Archivo: lib/services/photo_storage_service.dart. Fragmento de persistencia tras confirmar Guardar foto.

```
final saved = await source.copy(destination);
await _preferences.setString('photo_$productKey', saved.path);
return saved.path;
```

La ruta temporal se usa para pre visualizar. Al confirmar, la imagen se copia a collection_photos dentro del directorio privado de documentos de la app y se guarda su ruta. La clave productKey corresponde a producto.imagen. loadPhoto comprueba que el archivo exista antes de mostrarlo. Los errores de selección, lectura y escritura se comunican mediante el estado del Provider.

Canal nativo y notificaciones locales

Archivo: lib/services/notification_service.dart. Declaración del canal Android.

```
static const _channel = AndroidNotificationChannel(
  'collection_photos',
  'Fotos de colección',
  description: 'Confirmaciones de fotos guardadas en Relicarium II',
  importance: Importance.high,
);
```

initialize configura ic_stat_relicarium como icono y crea el canal con createNotificationChannel. La configuración Darwin pospone el permiso hasta usar la función. showPhotoSaved solicita autorización con la implementación de Android o iOS; si no se concede, devuelve false sin emitir la alerta (Flutter local notifications, s. f.).

Archivo: lib/providers/photo_provider.dart. Envío posterior al guardado correcto.

```
try {
  await NotificationService.instance.showPhotoSaved();
} catch (exception) {
  debugPrint('Foto guardada; fallo de notificacion: $exception');
}
```

Este bloque se ejecuta después de copiar y registrar la foto. Su catch es independiente del guardado: denegar notificaciones o fallar al emitirlas no elimina la imagen ni informa falsamente de un fallo al guardarla. La alerta usa el identificador 1, por lo que una confirmación posterior actualiza la anterior. El canal y los ajustes de Android determinan el sonido y la presentación emergente.

Sección IV

Evidencias de pruebas y repositorio público

Repositorio público: https://github.com/joseandresmendoza64/relicarium-flutter

Las pruebas manuales se realizaron en el emulador Android Pixel 10. Las capturas fueron aportadas por el estudiante durante la implementación; los registros textuales se conservaron junto con ellas. La Tabla 2 relaciona cada caso con su resultado. La ejecución automatizada se documenta aparte porque utiliza dependencias controladas.

Tabla 2

Resumen de QA realizado en Android

Caso	Resultado observado	Evidencia
Desconexión y reconexión	El banner aparece sin Internet y desaparece al recuperar Wi-Fi	Figura 4 y comprobación manual
Persistencia del catálogo	Guardado inicial y recuperación en otro proceso	Registros PID 3645 y 4406
Navegación offline	Se abre el detalle con sus datos	Figura 4
Cámara y permiso	Solicitud nativa, captura y foto guardada	Figura 5
Galería	Selector nativo y recuperación de foto por reliquia	Figura 6
Permiso de cámara bloqueado	Mensaje y acceso a Ajustes sin cerrar el catálogo	Figura 7
Notificación autorizada	Confirmación recibida al guardar una foto	Figura 8
Notificaciones desactivadas	La foto se guarda igualmente	Figura 9
Pruebas de widgets	Dos pruebas aprobadas con servicios controlados	Registro textual y código de test

Figura 4

Catálogo y detalle disponibles sin Internet

Catálogo offline



Detalle de Copa traslador



Nota. El estudiante confirmó la navegación sin Wi-Fi ni datos. Las ilustraciones son assets locales. Estas capturas se complementan con los registros de guardado y recuperación; no se usa solo la imagen del catálogo como prueba de persistencia.

Registro de la primera ejecución:

```
I/flutter (3645): Catalogo inicial guardado en almacenamiento local.
```

Registro de la segunda ejecución, después de cerrar y volver a abrir:

```
I/flutter (4406): Catalogo recuperado del almacenamiento local.
```

Figura 5

Solicitud de cámara y fotografía guardada

Permiso nativo de cámara

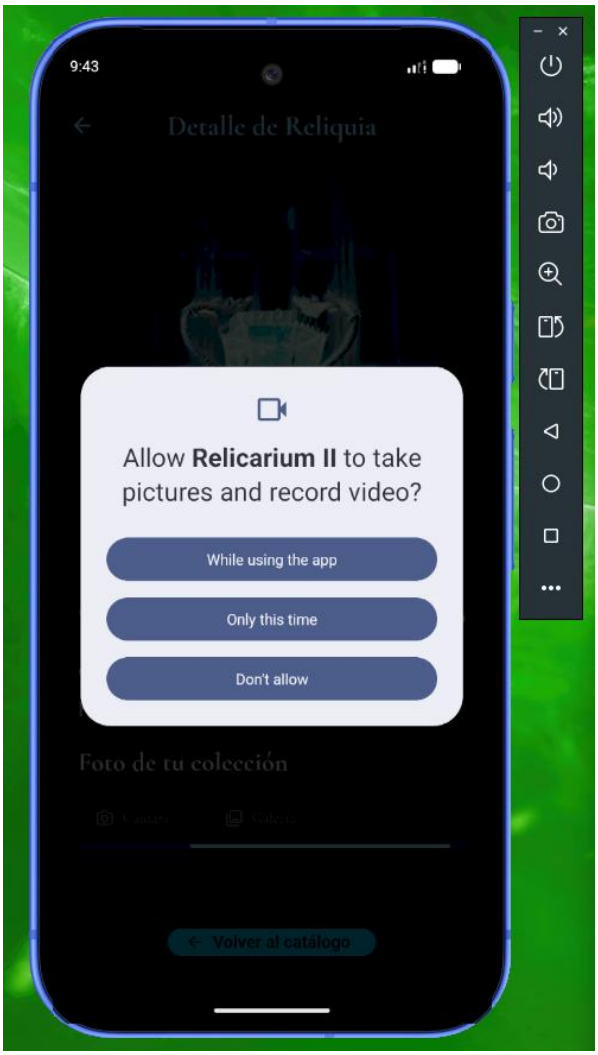
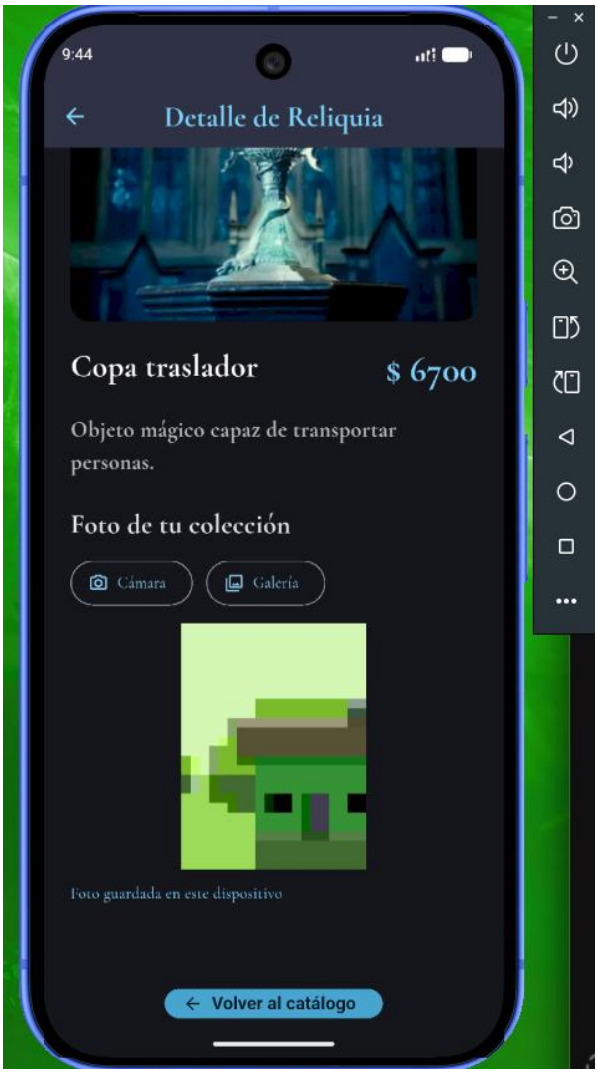


Foto en Copa traslador



Nota. Android solicita autorización durante el uso de la función. La foto verde corresponde a la escena de prueba de la cámara del emulador, no a una fotografía de un dispositivo físico. La pantalla derecha muestra el mensaje de guardado.

La captura se previsualiza antes de confirmarla. La ilustración original de la reliquia se conserva y la fotografía se presenta en Foto de tu colección, evitando reemplazar el recurso del catálogo.

Figura 6

Selección desde galería y recuperación de la foto personal

Selector nativo de fotos

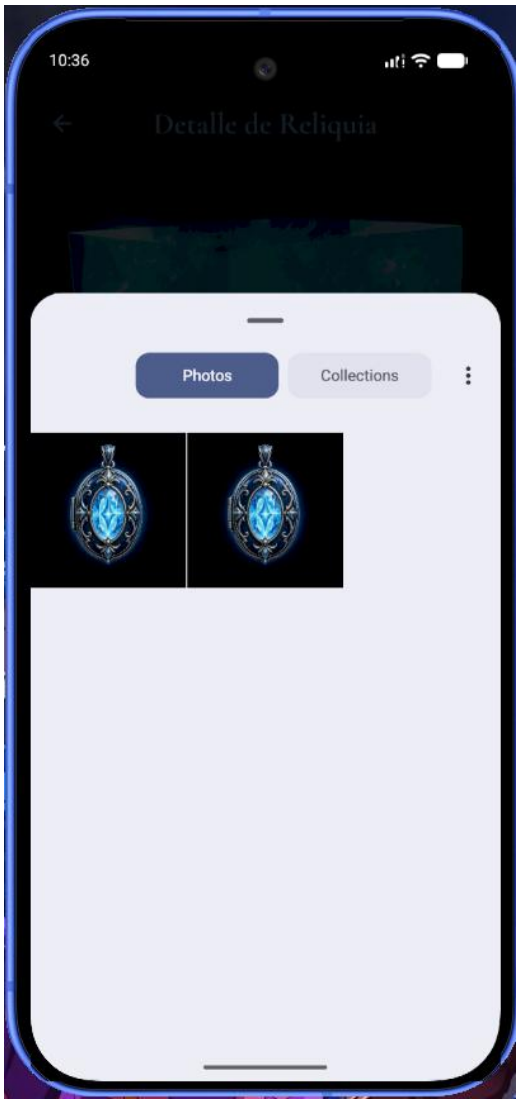
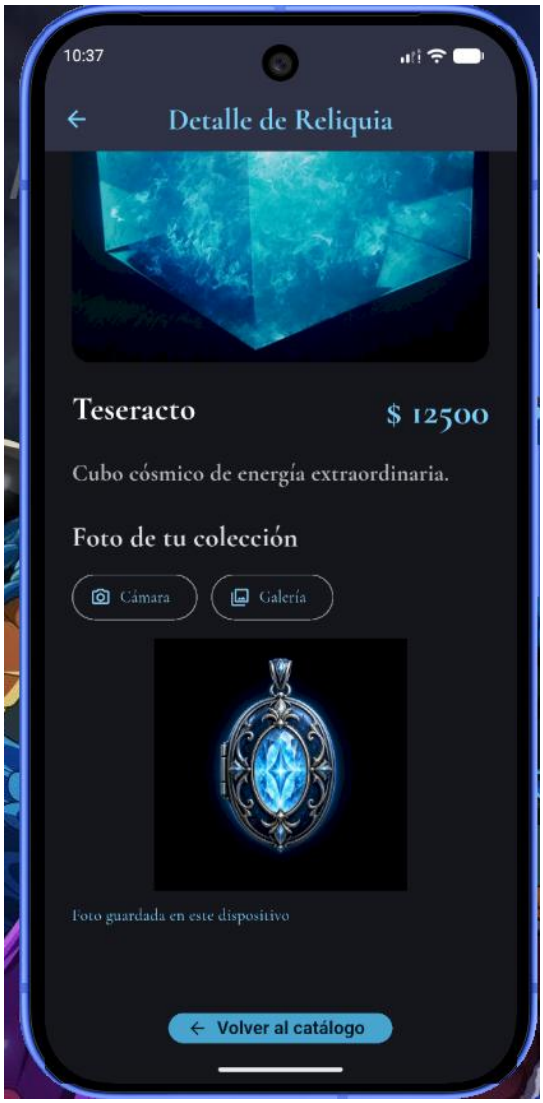


Imagen guardada en Teseracto



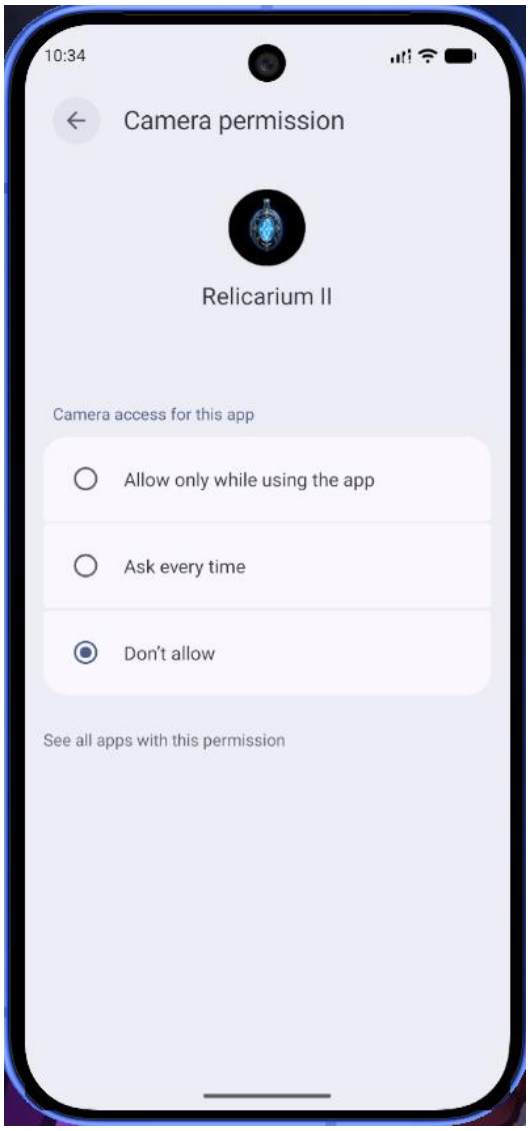
Nota. Se añadió una imagen de prueba al emulador para poblar su biblioteca. La imagen elegida corresponde al icono de Relicarium II. Las capturas documentan selección y conservación al volver al detalle; no acreditan por sí solas un reinicio completo del proceso.

La galería del teléfono y el almacenamiento privado de la app cumplen funciones distintas. Guardar foto no exporta la copia privada a la biblioteca pública. Esta decisión limita los permisos solicitados y mantiene la asociación de cada imagen con su reliquia.

Figura 7

Manejo del permiso de cámara bloqueado

Permiso No permitir



Respuesta de Relicarium II



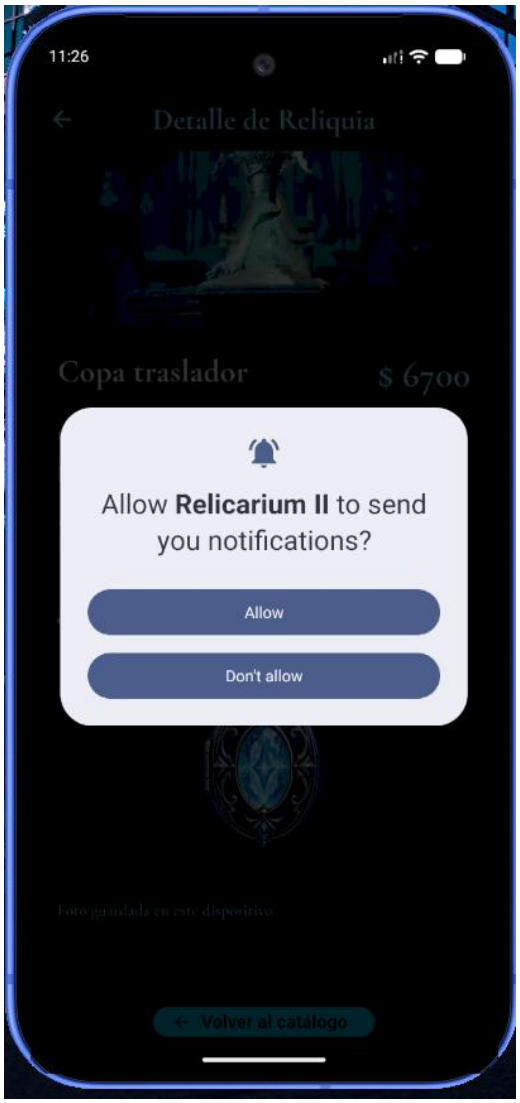
Nota. Se bloqueó la cámara desde los ajustes de Android. La app mostró una explicación y la acción Abrir ajustes. Se comprobó el acceso a la información de la aplicación desde esa acción.

La denegación no se trata como un cierre inesperado ni impide consultar el catálogo. El mensaje permite al usuario decidir si habilita el permiso. La alternativa Galería sigue disponible porque se gestiona mediante el selector nativo.

Figura 8

Autorización y recepción de una notificación local

Autorización de Android



Panel de notificaciones



Nota. La solicitud aparece al guardar una imagen. Después de autorizar, Android muestra Foto guardada y el mensaje de confirmación. La segunda captura corresponde al panel real del emulador; no es una representación diseñada de una notificación.

La notificación responde a un evento verificable del sistema de la aplicación: la copia y el registro correctos de una fotografía. No se necesita un servidor de mensajería ni un servicio push para esta alerta local.

Figura 9

El guardado permanece disponible al bloquear notificaciones

Notificaciones desactivadas

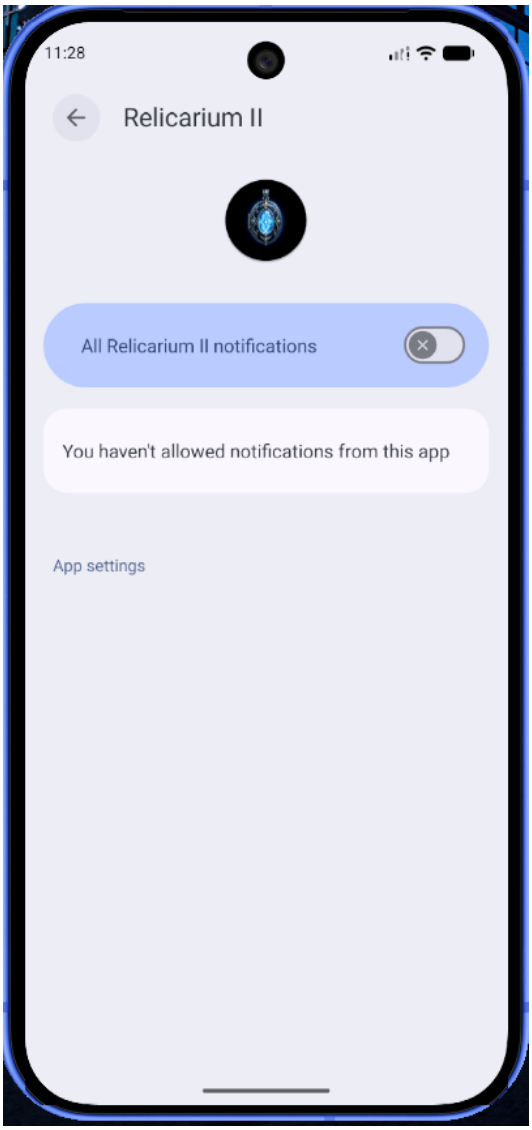


Foto guardada en Anticitera



Nota. El estudiante confirmó que el guardado continuó funcionando con las notificaciones desactivadas. La aplicación no condiciona la persistencia de la foto a la autorización para emitir alertas.

Este caso verifica la separación entre la función principal y una confirmación accesoria. La foto se conserva y la interfaz informa del guardado aunque Android no permita mostrar la notificación.

Pruebas automatizadas y trazabilidad

La prueba heredada buscaba RELICARIUM y no proporcionaba los nuevos Providers. Se actualizó para comprobar la nueva marca y el catálogo offline. Los dobles de prueba implementan las interfaces de servicio y repositorio, sin construir los plugins nativos. Se recibió el resultado textual siguiente tras la corrección:

```
flutter test
00:02 +2: All tests passed!
```

También se obtuvo No issues found con flutter analyze. Estas dos pruebas verifican estados controlados de interfaz; no constituyen pruebas automáticas de hardware ni de persistencia. La Tabla 3 muestra una selección de commits significativos de la segunda etapa, verificados en el historial local conectado al repositorio público.

Tabla 3

Selección del historial de la Actividad Integradora 2

Commit	Avance registrado
19c15ac	Nombre comercial Relicarium II
a238572	Identificador Android actualizado
1e4f6a4	Rediseño del catálogo y detalle
6930f5f	Icono y splash personalizados
2d1fec0	Monitoreo de Internet y banner offline
149a72e	Persistencia local del catálogo
fbcdbed	Cámara, galería, permisos y fotos por reliquia
0146cb1	Notificación local de guardado
f7dc57b	Pruebas de catálogo y estado offline
ce20513	Preparación de configuración iOS
294e2bd	README con instalación, arquitectura y pruebas

Nota. La rama develop conserva el desarrollo de la segunda etapa, integrado en master mediante el commit cbf2e6f. La etiqueta actividad-1-final conserva la referencia de la primera entrega. La numeración de commits no sustituye la revisión de su contenido.

Sección V

Conclusiones y lecciones aprendidas

La evolución de RELICARIUM a Relicarium II permitió conservar el concepto de artefactos legendarios y convertirlo en una experiencia de colección personal. El rebranding no se limitó al nombre: incorporó icono y splash propios, una paleta oscura y celeste y una navegación basada en tarjetas completas. El precio se mantiene en el detalle porque no es necesario mostrar todos los atributos en la pantalla principal para transferir el objeto seleccionado.

La separación en Providers, repositorio y servicios facilitó gestionar operaciones que terminan en momentos distintos. Escuchar streams, cancelar suscripciones y comprobar dispose evita actualizar componentes que ya no están activos. Al regresar del selector o de Ajustes, es necesario considerar el ciclo de vida del dispositivo, no solo el estado visual de una pantalla.

La persistencia se comprobó mediante guardado y lectura en ejecuciones distintas; la presencia de datos incluidos en el código, por sí sola, no habría demostrado ese requisito. También se comprobó que denegar una capacidad del dispositivo debe producir una respuesta comprensible y permitir continuar con las funciones disponibles. El guardado independiente de la notificación ejemplifica esa separación de responsabilidades.

El control de versiones conservó avances identificables y permitió distinguir problemas del producto de problemas del entorno. Durante el trabajo se diagnosticó agotamiento de memoria de Windows y se ajustaron la paginación y los límites de Gradle. Asimismo, se resolvieron incompatibilidades de dependencias revisando requisitos concretos, sin forzar versiones mediante overrides.

Límites y mejoras posteriores

La validación de esta entrega corresponde a Android. Permanecen como mejoras la verificación integral en iOS, la recuperación de una selección si Android termina el proceso durante la cámara o galería, la política de actualización de la copia local y la medición del tamaño final de las imágenes. El catálogo no se sincroniza con un servidor ni procesa pagos.

Declaración de asistencia de inteligencia artificial

Durante esta actividad utilicé ChatGPT y Codex como apoyo para interpretar los requisitos, diseñar la identidad visual, generar y explicar código, diagnosticar errores, actualizar pruebas y elaborar documentación. La asistencia incluyó partes sustanciales de la implementación y de las explicaciones técnicas. El icono del relicario también se generó con IA.

Realicé la integración de los cambios y las comprobaciones en mi entorno, y aporté las capturas y los registros de ejecución utilizados en este informe. Las decisiones de conservar la raíz Relicarium, usar el nombre Relicarium II y priorizar la paleta oscura y celeste corresponden a mis preferencias expresadas durante el desarrollo. Los diagramas son explicaciones del código y no se presentan como capturas del dispositivo.

Referencias

Baseflow. (s. f.). *permission_handler* (versión 12.0.1) [Paquete de software]. Pub.dev. https://pub.dev/packages/permission_handler/versions/12.0.1

Flutter. (s. f.-a). *image_picker* (versión 1.2.4) [Paquete de software]. Pub.dev. https://pub.dev/packages/image_picker/versions/1.2.4

Flutter. (s. f.-b). *shared_preferences* (versión 2.5.6) [Paquete de software]. Pub.dev. https://pub.dev/packages/shared_preferences/versions/2.5.6

Flutter. (s. f.-c). *Simple app state management*. https://docs.flutter.dev/data-and-backend/state-mgmt/simple

Flutter Community. (s. f.). *connectivity_plus* (versión 7.3.2) [Paquete de software]. Pub.dev. https://pub.dev/packages/connectivity_plus/versions/7.3.2

Flutter local notifications (versión 22.3.3) [Paquete de software]. (s. f.). Pub.dev. https://pub.dev/packages/flutter_local_notifications/versions/22.3.3

Internet connection checker plus (versión 3.1.2) [Paquete de software]. (s. f.). Pub.dev. https://pub.dev/packages/internet_connection_checker_plus/versions/3.1.2